

Supplementary Material for Cost- and SLA-Bounded Orchestration for LLM-Agent Tool/Service Composition

Anonymous submission, ICSOC 2026 research track

This supplement provides the full proof details and secondary ablation studies referenced in the main paper. It is intended to accompany main.tex and supplement.tex in the same submission source tree.

S1. Problem Scope and Schema Pruning

CBSLAO asks an agent policy to select a sequence of tool invocations that maximises expected utility subject to chance-constrained budget and deadline requirements. The paper distinguishes non-adaptive, semi-adaptive, and adaptive policies. The online regret theorem competes with the best chance-constrained non-adaptive policy, because tight regret against the fully adaptive two-resource chance-constrained oracle is left as an open problem.

Proposition S1 (completeness of offline pruning). Let $cl(q)$ start from the query types and iteratively add every output schema Out_i whose input schema In_i is already contained in $cl(q)$. If a feasible non-adaptive policy invokes tool s_i , then s_i belongs to the pruned candidate set $S_q = \{s_i : In_i \subset cl(q)\}$.

Proof. Order the calls in the non-adaptive plan. Initially the first call can only require query-available types, hence its outputs are added by the closure rule. Inductively, before call j every type produced by calls $1, \dots, j-1$ is already in $cl(q)$, so any input required by call j is contained in $cl(q)$, and the closure rule includes the tool. Therefore every tool in the plan lies in S_q .

S2. Hardness and Approximability

Theorem S2 (NP-hardness). CBSLAO-DEC is NP-hard even with deterministic costs, no latency constraint, additive utility, and non-adaptive policies. Reduce from 0/1 Knapsack. For each knapsack item i with weight w_i and value v_i create a tool with deterministic cost w_i , zero latency, and utility contribution v_i . Set the budget to W and the target utility to V . Because costs are deterministic, the chance constraint is exactly the knapsack capacity. A policy reaches the target utility within budget if and only if the original knapsack instance has a feasible subset of value at least V .

Theorem S3 ($1 - 1/e$ barrier). Unless $P = NP$, no polynomial-time algorithm achieves a $(1 - 1/e + \epsilon)$ -approximation for the offline non-adaptive monotone-submodular case. Reduce from Max-k-Coverage: each set becomes a unit-cost tool, each universe element becomes an output type, and utility is the normalised size of the union of covered types. Selecting at most k tools is exactly selecting at most k sets. Any stronger CBSLAO approximation would imply a stronger Max-k-Coverage approximation, contradicting Feige's threshold.

Theorem S4 (matching upper bound). For deterministic costs and saturating-additive utility $U(A) = g(\sum_i u_i)$, where g is monotone, non-decreasing, concave, and $g(0) = 0$, the set function is monotone submodular. Sviridenko's modified greedy algorithm for monotone submodular maximisation under one knapsack constraint returns a $(1 - 1/e)$ -approximation in polynomial time. Thus the lower bound is tight up to the usual epsilon slack.

Lemma S5 (adaptivity gap). CBSLAO inherits a worst-case adaptivity gap of at least $4/3$ from stochastic knapsack by setting latency to zero and using additive value. Existing stochastic-knapsack results also give a constant-factor ceiling for the resource-constrained regime. In the simulator used in the paper, the empirical non-adaptive to informed-knapsack utility ratio is 1.00 ± 0.05 , so this worst-case gap is not visible in the reported workload family.

S3. Regret Proof for CBUC

Theorem S6. Under bounded sub-Gaussian cost, latency, and utility noise; a positive minimum mean cost $c_{\min}$; and a fixed chance level δ in $(0, 1/2]$, CBUC with empirical-Bernstein confidence widths achieves, with probability at least $1 - \delta$, regret $C_1 \sqrt{KT \log T} + C_2 \sum \sigma_i \sqrt{T} + C_3$ against the best chance-constrained non-adaptive policy. Hence $REG(T)$ is $\tilde{O}(\sqrt{KT})$.

L1. Coupled concentration.

For each arm, view the observation as the vector $X=(\text{cost}, \text{latency}, \text{utility})$ in $[0,M]^3$. Applying empirical Bernstein component-wise at level $\delta/3$ and taking the infinity norm gives a radius bounded by $\sqrt{2 V_{\max} \log(6/\delta') / N} + 7 M \log(6/\delta') / (3(N-1))$. Union-bounding over K arms and T rounds yields a uniform confidence event with failure probability at most $\delta/3$. Keeping the three coordinates coupled inside the vector event improves the leading constant relative to a fully decoupled three-resource union bound.

L2. Feasibility preservation.

CBUC shrinks the nominal budget and deadline by $z \sigma \sqrt{T}$. On the concentration event, each accepted call has estimated cost and latency plus its confidence margin below the remaining shrunken resource. The sub-Gaussian tail bound then implies that realised aggregate cost exceeds the nominal budget with probability at most $\delta/3$, and the same argument applies to latency. A union bound gives the stated chance-constraint guarantee.

L3. Lagrangian reduction.

On the shrunken instance, the chance-constrained primal can be treated as a mean-feasible two-resource knapsack. Introducing non-negative dual variables for budget and deadline gives the linearised score $L_i(\lambda)=\mu_i-\lambda_B c_i-\lambda_D \ell_i$. At the optimal dual point, regret in this modified instance controls regret against the non-adaptive chance-constrained oracle, up to the 0/1 rounding gap.

L4. Bandits-with-knapsacks regret.

Conditioned on L1 and L2, the modified problem satisfies the standard inputs for a two-resource BwK analysis. Applying the BwK regret bound gives $O(\sqrt{KT \log T})$ Lagrangian regret. Translating from the shrunken budget back to the original budget adds at most a Lipschitz resource correction proportional to $z \sigma \sqrt{T}$. Combining the events yields the theorem's probability and regret statement.

Heavy-tailed costs. When the Pareto tail has infinite or unstable variance, the sub-Gaussian theorem does not apply. The supplement's CVaR variant replaces the Gaussian margin with a uniformly consistent empirical-CVaR estimate, preserving the chance constraint with an additional $\sqrt{\log T}$ factor in the correction term. The Pareto-1.5 rows in the paper should therefore be read as empirical robustness stress tests, not as theorem-covered evidence.

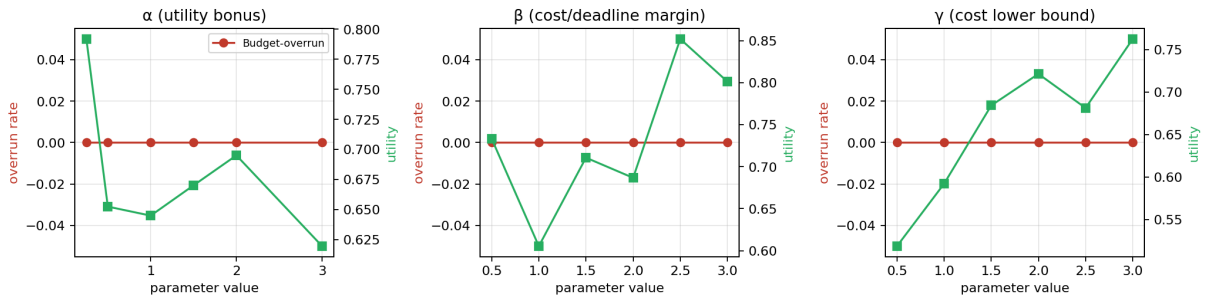
S4. Secondary Ablations

A1. Sensitivity to alpha, beta, gamma.

A one-at-a-time sweep around the defaults $\alpha=1.0$, $\beta=1.5$, $\gamma=1.5$ uses $K=50$, $\rho=0.5$, tail index 2.0, sparsity 0.2, and 40 seeds per setting. Budget overrun is 0.000 throughout the grid; utility varies from 0.519 to 0.852.

Param	Value	Overrun	Utility
alpha	0.25	0.000	0.792
alpha	0.50	0.000	0.652
alpha	1.00	0.000	0.645
alpha	1.50	0.000	0.670
alpha	2.00	0.000	0.695
alpha	3.00	0.000	0.620
beta	0.50	0.000	0.733
beta	1.00	0.000	0.605
beta	1.50	0.000	0.711
beta	2.00	0.000	0.687
beta	2.50	0.000	0.852
beta	3.00	0.000	0.801
gamma	0.50	0.000	0.519
gamma	1.00	0.000	0.592
gamma	1.50	0.000	0.684
gamma	2.00	0.000	0.721
gamma	2.50	0.000	0.681
gamma	3.00	0.000	0.762

CBUC sensitivity to α , β , γ ($K=50$, $\rho=0.5$, tail=2.0, $n=40$)



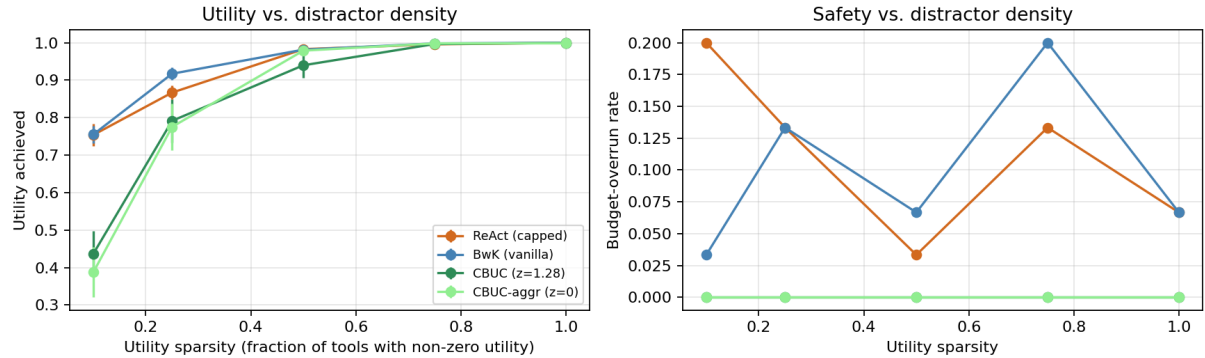
A2. Utility-dense distractors.

This ablation varies the fraction of useful tools from 0.10 to 1.00. CBUC maintains zero budget overrun at all densities, while ReAct-capped and BwK-vanilla reach overrun rates up to 20 percent.

Policy	Sparsity	Overrun	Utility
ReAct-capped	0.10	0.200	0.754
ReAct-capped	0.25	0.133	0.867

Policy	Sparsity	Overrun	Utility
ReAct-capped	0.50	0.033	0.983
ReAct-capped	0.75	0.133	0.996
ReAct-capped	1.00	0.067	1.000
BwK-vanilla	0.10	0.033	0.755
BwK-vanilla	0.25	0.133	0.917
BwK-vanilla	0.50	0.067	0.981
BwK-vanilla	0.75	0.200	0.998
BwK-vanilla	1.00	0.067	1.000
CBUC	0.10	0.000	0.437
CBUC	0.25	0.000	0.791
CBUC	0.50	0.000	0.940
CBUC	0.75	0.000	0.996
CBUC	1.00	0.000	0.999

Effect of distractor density ($K=50$, $\rho=0.5$, $\text{tail}=2.0$, $n=30$)

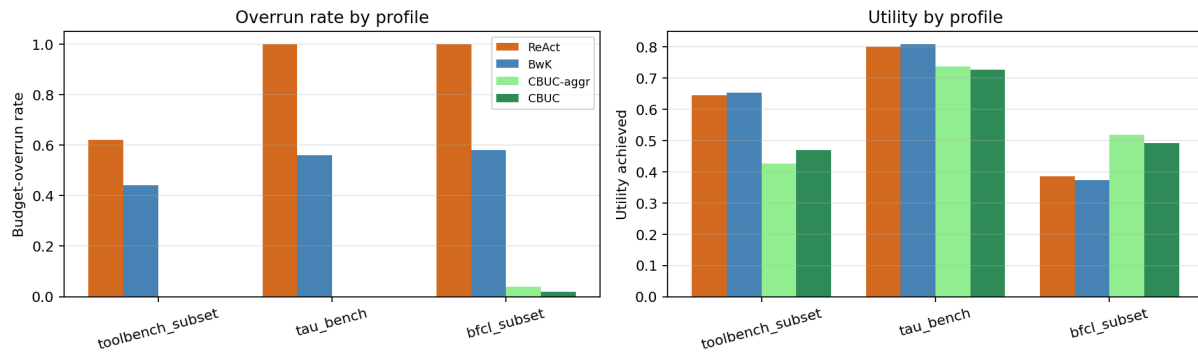


A3. Calibrated benchmark-style replay.

The replay ablation parameterises the simulator to published summary statistics from ToolBench-like, tau-bench-like, and BFCL-like profiles. These are calibrated simulations, not live API trace replays. CBUC achieves at most 2 percent overrun across all profiles; the two baselines reach 44 to 100 percent overrun.

Profile	Policy	Overrun	Utility	Regret
ToolBench	ReAct-capped	0.620	0.646	+0.324
ToolBench	BwK-vanilla	0.440	0.653	+0.318
ToolBench	CBUC-aggr	0.000	0.427	+0.543
ToolBench	CBUC	0.000	0.469	+0.501
tau-bench	ReAct-capped	1.000	0.802	+0.135
tau-bench	BwK-vanilla	0.560	0.810	+0.127
tau-bench	CBUC-aggr	0.000	0.738	+0.199
tau-bench	CBUC	0.000	0.728	+0.208
BFCL	ReAct-capped	1.000	0.387	+0.491
BFCL	BwK-vanilla	0.580	0.374	+0.504
BFCL	CBUC-aggr	0.040	0.518	+0.360
BFCL	CBUC	0.020	0.492	+0.386

Calibrated replay (NOT a live-trace replay; see §caveat)



S5. Reproducibility Notes

The headline results are generated by code/cbslao_sim.py and code/analyze.py. The secondary ablations are generated by code/ablations.py and stored under out/ablation_*.csv in the repository root. The submission source tree also contains supplement.tex, which is the LaTeX source version of this material for LNCS builds.